

KN-STORE

Plataforma de Comercio Electrónico para Calzado

Documento de Especificación de Requerimientos de Software

(SRS — IEEE 830 adaptado)

Versión	1.0
Fecha	Abril de 2026
Estado	Documento base de especificación
Audiencia	Equipo de desarrollo, QA, dirección de proyecto
Estándar	IEEE 830 (adaptado)

1. Introducción

1.1 Propósito del documento

El presente documento tiene por objeto especificar de manera formal, completa y verificable los requerimientos funcionales y no funcionales del sistema KN-Store, una plataforma de comercio electrónico orientada a la comercialización de calzado. La especificación está dirigida al equipo de desarrollo, al equipo de aseguramiento de calidad (QA), a la dirección del proyecto y a cualquier interesado responsable de validar, mantener o evolucionar el sistema.

El documento describe el comportamiento esperado del software, sus interfaces, las restricciones bajo las cuales debe operar y los criterios mediante los cuales se considerará que cada requerimiento ha sido satisfecho.

1.2 Alcance del sistema

KN-Store es una plataforma web de comercio electrónico construida sobre el stack MERN (MongoDB, Express, React, Node.js), cuyo dominio principal es la gestión de un catálogo de calzado. El sistema cubre los siguientes ámbitos funcionales:

- Autenticación y autorización de usuarios mediante tokens JWT con vigencia de 30 días.
- Gestión integral del catálogo de productos, incluyendo categorías, subcategorías, variantes (talla, color, SKU, stock, precios) y descuentos con vigencia temporal.
- Administración de usuarios con tres roles diferenciados: Admin, Manager y Client.
- Consulta pública del catálogo con búsqueda full-text, filtros por precio, marca, etiquetas y disponibilidad de stock, así como paginación.
- Panel administrativo (back-office) para operaciones CRUD sobre los módulos de usuarios, categorías, subcategorías y productos.
- Eliminación lógica (soft delete) y, exclusivamente para administradores, eliminación física definitiva.
- Gestión de direcciones de envío del usuario, con marca de dirección predeterminada.
- Carrito de compras persistente por usuario, modelado como un pedido en estado CART.
- Gestión de pedidos a través de un ciclo de vida explícito: CART → PENDING → CONFIRMED → SHIPPED → DELIVERED, con la posibilidad de transición a CANCELLED.
- Procesamiento de pagos con estados PENDING, COMPLETED, FAILED y REFUNDED.
- Gestión de envíos con número de seguimiento, método de envío y estados PENDING, SHIPPED, IN_TRANSIT, DELIVERED y RETURNED.
- Generación automática de facturas con referencia única tras la confirmación del pago.

Los módulos de direcciones, pedidos, pagos, envíos y facturación están especificados conforme al diagrama de clases del sistema y forman parte del alcance funcional definitivo. Su estado de implementación se indica explícitamente en cada requerimiento mediante las etiquetas [Implementado], [Parcial] o [Pendiente]. Quedan fuera del alcance del presente documento la

integración con pasarelas de pago externas concretas, la facturación electrónica con autoridad fiscal y la integración con operadores logísticos específicos; estos elementos serán objeto de especificaciones complementarias.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
API	Application Programming Interface. Conjunto de endpoints HTTP expuestos por el backend.
CRUD	Create, Read, Update, Delete. Operaciones básicas sobre una entidad persistente.
JWT	JSON Web Token. Estándar abierto (RFC 7519) para la transmisión segura de información entre partes mediante objetos JSON firmados.
MERN	MongoDB, Express, React, Node.js. Conjunto de tecnologías sobre el cual está construido el sistema.
MongoDB	Sistema de gestión de bases de datos documental no relacional utilizado como repositorio de persistencia.
Mongoose	Biblioteca ODM (Object Data Modeling) para Node.js que provee modelado de esquemas y validaciones sobre MongoDB.
SKU	Stock Keeping Unit. Identificador único de una variante específica de un producto.
Slug	Cadena legible y única derivada del nombre de un producto, utilizada en URLs amigables.
Soft delete	Eliminación lógica. La entidad permanece en la base de datos con el atributo active marcado en falso, ocultándose de los listados públicos.
SRS	Software Requirements Specification. Documento de especificación de requerimientos.
RF	Requerimiento Funcional.
RNF	Requerimiento No Funcional.
Bearer token	Esquema de autenticación HTTP que transporta el JWT en la cabecera Authorization.

1.4 Referencias

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.
- RFC 7519: JSON Web Token (JWT).
- Documentación oficial de Next.js 15 (App Router).
- Documentación oficial de Express 5 y Mongoose 9.

- Documentación interna del proyecto: Docs/01-ANALYSIS y Docs/02-DESING (levantamiento de información, BPMN, casos de uso, diagrama de clases, diagrama de despliegue, wireframes).

1.5 Resumen del documento

El documento se estructura en siete secciones principales. La sección 2 ofrece una descripción general del producto, sus usuarios y el entorno operacional. La sección 3 detalla los requerimientos funcionales agrupados por módulo. La sección 4 establece los requerimientos no funcionales con criterios verificables. La sección 5 describe las interfaces externas del sistema. La sección 6 enuncia las restricciones y reglas de negocio críticas. Finalmente, la sección 7 presenta una matriz de trazabilidad entre requerimientos y componentes implementados.

2. Descripción General del Sistema

2.1 Perspectiva del producto

KN-Store es un producto de software autocontenido compuesto por dos aplicaciones desplegables independientemente: un servidor backend basado en Node.js y Express que expone una API REST sobre HTTP, y una aplicación frontend basada en Next.js 15 que consume dicha API. La persistencia se realiza sobre una base de datos MongoDB, accedida mediante el ODM Mongoose.

El sistema interactúa con los siguientes elementos externos:

- Navegadores web modernos como cliente final del usuario.
- Base de datos MongoDB como servicio de persistencia.
- Almacenamiento local del navegador (localStorage) para la persistencia del token de sesión.

La arquitectura sigue un patrón de capas en el backend (Ruta → Middleware de validación → Controlador → Modelo) y un patrón de composición basado en Server Components y Client Components en el frontend, con gestión de estado de autenticación mediante React Context.

2.2 Funciones del producto

A alto nivel, el sistema provee las siguientes funciones principales:

- Registro y autenticación de usuarios con persistencia de sesión por tokens JWT.
- Consulta pública del catálogo de productos con búsqueda, filtros y paginación.
- Administración del catálogo (categorías, subcategorías y productos) por parte de Managers y Administradores.
- Administración de usuarios y asignación de roles por parte de Administradores.
- Gestión del perfil personal por parte de cualquier usuario autenticado.
- Cálculo automático de precios finales y márgenes de utilidad sobre productos con descuentos vigentes.
- Eliminación lógica universal y eliminación física restringida al rol Admin.
- Gestión de direcciones del usuario con marcado de dirección predeterminada.
- Operación de carrito de compras: agregar productos, modificar cantidades, calcular totales y persistir entre sesiones.
- Confirmación de pedidos (checkout) con asignación de dirección de envío y descuento de stock.
- Procesamiento de pagos con registro de método, monto y estado, y soporte de reembolsos.
- Asignación y seguimiento de envíos con número de tracking y actualización de estado.
- Generación automática de facturas con referencia única al completarse el pago.

2.3 Características y clases de usuarios

Rol	Descripción	Permisos principales
Admin	Administrador con control total del sistema.	CRUD completo en todos los módulos. Único rol autorizado para eliminación física. Único rol que puede crear usuarios con cualquier rol.
Manager	Gestor responsable del catálogo de productos.	Crear y editar categorías, subcategorías y productos. No puede gestionar usuarios ni realizar eliminaciones físicas.
Client	Usuario comprador final del sistema.	Consultar el catálogo público, gestionar su propio perfil, consultar el detalle de productos.
Anónimo	Visitante no autenticado.	Consultar el catálogo público, registrarse, iniciar sesión.

Nota: existe un rol histórico denominado Provider que ha sido migrado a Manager. El sistema mantiene compatibilidad con datos legacy donde aplique, pero ningún flujo nuevo emite el rol Provider.

2.4 Entorno operacional

Componente	Tecnología	Versión
Frontend — framework	Next.js (App Router)	15.3.2
Frontend — librería UI	React	19.2.4
Frontend — animaciones	Framer Motion	12.38.0
Frontend — animaciones avanzadas	GSAP + Lenis	3.14.2 / 1.3.21
Backend — runtime	Node.js (LTS)	20.x o superior
Backend — framework HTTP	Express	5.2.1
Persistencia	MongoDB + Mongoose	9.3.0 (ODM)
Autenticación	jsonwebtoken	9.0.3
Hashing de contraseñas	bcryptjs	3.0.3
Validación de entradas	express-validator	7.3.1
Tipografías	Sora (display) y Space Mono (monoespaciada)	—
Estilado	CSS personalizado con tokens de diseño (--kn-*, --sp-*, --dur-*)	—

El backend escucha por defecto en el puerto 3000 y expone su raíz en /api. El frontend consume la API a través de rewrites configurados en next.config.js para evitar problemas de CORS en

desarrollo. Ambos componentes son compatibles con sistemas operativos Linux, macOS y Windows en sus versiones soportadas para Node.js LTS.

2.5 Restricciones de diseño e implementación

- La autenticación se realiza exclusivamente mediante JWT con vigencia de 30 días, transportados en la cabecera Authorization bajo el esquema Bearer.
- Las contraseñas se almacenan exclusivamente como hash bcrypt; en ningún caso el sistema persiste, registra ni retorna la contraseña en claro.
- El estilado no utiliza Tailwind ni librerías de componentes UI: se emplea CSS personalizado y un sistema propio de tokens de diseño.
- Las respuestas de la API siguen un formato estandarizado: {success: true, data: ...} o {success: false, message: ...}.
- La eliminación lógica es la operación por defecto. La eliminación física es una operación adicional, posterior y restringida al rol Admin.
- El registro público a través de /api/users/register sólo puede crear usuarios con rol Client; la creación de Managers y Admins requiere autenticación previa con rol Admin.

2.6 Suposiciones y dependencias

- Se asume que los usuarios finales acceden al sistema desde navegadores modernos con soporte para ES2020+ y JavaScript habilitado.
- Se asume conectividad a una instancia de MongoDB accesible desde el backend mediante una cadena de conexión configurada por variable de entorno.
- Se asume que las variables de entorno sensibles (secreto JWT, credenciales de base de datos) se gestionan fuera del código fuente.
- El sistema depende de la disponibilidad de la base de datos MongoDB; en su ausencia, las operaciones de lectura y escritura no podrán completarse.

2.7 Modelo de datos consolidado

El siguiente cuadro resume las entidades del sistema conforme al diagrama de clases del proyecto. Las entidades se agrupan por dominio funcional y se indican sus campos principales y las restricciones más relevantes.

Entidad	Dominio	Campos principales	Restricciones
users	Usuarios y direcciones	id, email, password, name, last_name, role (ADMIN/MANAGER/CLIENT), active, timestamps	email único; password hashado
address	Usuarios y direcciones	id, user_id, street, number, city, postal_code, is_default, created_at	FK a users; máx. una predeterminada por usuario
category	Catálogo	id, name, description, active, created_at	name único

Entidad	Dominio	Campos principales	Restricciones
subcategory	Catálogo	id, category_id, name, description, active, created_at	FK a category
product	Catálogo	id, subcategory_id, name, price, size, stock, brand, active, timestamps	FK a subcategory; price DECIMAL(10,2)
orders	Órdenes y detalle	id, user_id, address_id, order_date, status (CART/PENDING/CONFIRMED/SHIPPED/DELIVERED/CANCELLED), total, updated_at	FK a users y address; total DECIMAL(12,2) a lo sumo un pedido CART por usuario
order_item	Órdenes y detalle	id, order_id, product_id, quantity, unit_price, subtotal (generado)	FK a orders y product; subtotal = quantity × unit_price
payment	Pagos, envíos y facturación	id, order_id, amount, method, status (PENDING/COMPLETED/FAILED/REFUNDED), paid_at, created_at	FK a orders; amount DECIMAL(12,2) amount = orders.total al momento de creación
shipment	Pagos, envíos y facturación	id, order_id, tracking_number, shipping_method, status (PENDING/SHIPPED/IN_TRANSIT/DELIVERED/RETURNED), shipped_at, delivered_at, created_at	FK a orders; a lo sumo un envío por pedido
bill	Pagos, envíos y facturación	id, order_id, reference, total, created_at	FK a orders; reference único total DECIMAL(12,2) generación idempotente

Nota sobre el modelado de productos: el diagrama de clases representa el producto con campos atómicos (price, size, stock). La implementación actual del módulo de catálogo en MongoDB utiliza un esquema enriquecido que incorpora arreglos embebidos de variantes y descuentos; ambas representaciones son funcionalmente equivalentes para los flujos especificados en este documento, y la traducción entre una y otra forma parte del trabajo técnico de implementación de los módulos pendientes.

3. Requerimientos Funcionales

Esta sección detalla los requerimientos funcionales del sistema, agrupados por módulo. Cada requerimiento sigue un formato uniforme con identificador, descripción, actor responsable, condiciones previas y posteriores, prioridad y estado de implementación.

3.1 Módulo de Autenticación y Sesión

RF-001 — Registro de usuario nuevo

Identificador	RF-001
Nombre	Registro de usuario nuevo
Descripción	El sistema debe permitir a un visitante anónimo registrarse mediante el endpoint POST /api/users/register, proporcionando nombre, correo electrónico y contraseña. El sistema validará el formato del correo, la fortaleza mínima de la contraseña y la unicidad del correo. El usuario será creado únicamente con rol Client.
Actor	Visitante anónimo
Precondición	El correo proporcionado no debe estar registrado previamente en el sistema.
Postcondición	Se crea un nuevo registro en la colección de usuarios con rol Client, contraseña hasheada con bcrypt y atributo active en verdadero. Se retorna un token JWT válido por 30 días.
Prioridad	Alta
Estado	Implementado

RF-002 — Inicio de sesión

Identificador	RF-002
Nombre	Inicio de sesión
Descripción	El sistema debe permitir a cualquier usuario registrado iniciar sesión mediante el endpoint POST /api/users/login proporcionando correo y contraseña. El sistema verificará la coincidencia del hash y, en caso afirmativo, emitirá un token JWT.
Actor	Usuario registrado (cualquier rol)
Precondición	El usuario debe existir en el sistema con atributo active en verdadero.
Postcondición	Se retorna un token JWT con vigencia de 30 días que el cliente persiste en localStorage. El estado de autenticación del frontend se actualiza vía AuthContext.
Prioridad	Alta

Estado	Implementado
---------------	--------------

RF-003 — Cierre de sesión

Identificador	RF-003
Nombre	Cierre de sesión
Descripción	El sistema debe permitir al usuario autenticado cerrar su sesión mediante el endpoint POST /api/users/logout. La operación invalidará localmente el token y limpiará el almacenamiento del navegador.
Actor	Usuario autenticado
Precondición	El usuario debe poseer una sesión activa con un token JWT válido.
Postcondición	El token se elimina de localStorage y el AuthContext refleja un estado no autenticado. El cliente queda redirigido al estado público.
Prioridad	Alta
Estado	Implementado

RF-004 — Verificación automática de sesión

Identificador	RF-004
Nombre	Verificación automática de sesión
Descripción	Al cargar el frontend, el sistema debe verificar la existencia y vigencia del token JWT en localStorage y, si es válido, restablecer el estado de autenticación del usuario sin requerir una nueva autenticación manual.
Actor	Sistema (frontend)
Precondición	Existe un token JWT en localStorage.
Postcondición	Si el token es válido y vigente, el AuthContext expone los datos del usuario; en caso contrario, el token se descarta y el usuario se considera anónimo.
Prioridad	Alta
Estado	Implementado

RF-005 — Protección de rutas mediante middleware de autorización

Identificador	RF-005
Nombre	Protección de rutas mediante middleware de autorización

Descripción	Toda solicitud a rutas privadas o restringidas debe ser interceptada por un middleware que valide el token JWT y verifique que el rol del usuario corresponda al permitido por la ruta.
Actor	Sistema (backend)
Precondición	La solicitud entrante incluye una cabecera Authorization con un token Bearer.
Postcondición	Si el token es válido y el rol cumple con la política de la ruta, la solicitud continúa al controlador. En caso contrario, el sistema responde con código HTTP 401 (no autenticado) o 403 (no autorizado).
Prioridad	Alta
Estado	Implementado

3.2 Módulo de Gestión de Usuarios (Administración)

RF-006 — Listar todos los usuarios

Identificador	RF-006
Nombre	Listar todos los usuarios
Descripción	El sistema debe permitir al rol Admin obtener la lista de todos los usuarios registrados mediante GET /api/users. La respuesta excluye el campo password de cada documento.
Actor	Admin
Precondición	El usuario debe estar autenticado con rol Admin.
Postcondición	Se retorna un arreglo de usuarios con todos sus campos públicos.
Prioridad	Alta
Estado	Implementado

RF-007 — Consultar usuario por identificador

Identificador	RF-007
Nombre	Consultar usuario por identificador
Descripción	El sistema debe permitir al rol Admin consultar el detalle de cualquier usuario mediante GET /api/users/:id.
Actor	Admin
Precondición	El usuario consultado debe existir; el solicitante debe ser Admin.
Postcondición	Se retorna el documento del usuario solicitado, sin el campo password.

Prioridad	Alta
Estado	Implementado

RF-008 — Crear usuario con rol arbitrario

Identificador	RF-008
Nombre	Crear usuario con rol arbitrario
Descripción	El sistema debe permitir al rol Admin crear nuevos usuarios mediante POST /api/users especificando explícitamente el rol (Admin, Manager o Client). Esta operación se diferencia del registro público que sólo crea usuarios con rol Client.
Actor	Admin
Precondición	El correo proporcionado no debe existir; el solicitante debe ser Admin.
Postcondición	Se crea el usuario con el rol indicado, contraseña hasheada y atributo active en verdadero.
Prioridad	Alta
Estado	Implementado

RF-009 — Actualizar datos de un usuario

Identificador	RF-009
Nombre	Actualizar datos de un usuario
Descripción	El sistema debe permitir al rol Admin modificar los atributos de cualquier usuario, incluido su rol y estado activo, mediante PUT /api/users/:id.
Actor	Admin
Precondición	El usuario objetivo debe existir; el solicitante debe ser Admin.
Postcondición	El documento del usuario se actualiza con los valores provistos. Si la contraseña fue modificada, se rehashea antes de persistirse.
Prioridad	Alta
Estado	Implementado

RF-010 — Eliminar usuario (lógica y física)

Identificador	RF-010
Nombre	Eliminar usuario (lógica y física)

Descripción	El sistema debe permitir al rol Admin eliminar usuarios mediante DELETE /api/users/:id. La operación realiza eliminación lógica por defecto (active = false). Una segunda invocación sobre un usuario ya inactivo produce su eliminación física definitiva.
Actor	Admin
Precondición	El usuario objetivo debe existir; el solicitante debe ser Admin.
Postcondición	En la primera invocación, el atributo active del usuario pasa a falso. En la segunda invocación, el documento se elimina permanentemente de la base de datos.
Prioridad	Alta
Estado	Implementado

3.3 Módulo de Gestión de Categorías

RF-011 — Listar categorías públicamente

Identificador	RF-011
Nombre	Listar categorías públicamente
Descripción	El sistema debe exponer la lista de categorías activas mediante GET /api/categories sin requerir autenticación.
Actor	Cualquier usuario (incluido anónimo)
Precondición	Ninguna.
Postcondición	Se retorna el arreglo de categorías con atributo active igual a verdadero.
Prioridad	Alta
Estado	Implementado

RF-012 — Consultar categoría por identificador

Identificador	RF-012
Nombre	Consultar categoría por identificador
Descripción	El sistema debe permitir consultar el detalle de una categoría específica mediante GET /api/categories/:id.
Actor	Cualquier usuario
Precondición	La categoría debe existir.
Postcondición	Se retorna el documento de la categoría solicitada.

Prioridad	Media
Estado	Implementado

RF-013 — Crear categoría

Identificador	RF-013
Nombre	Crear categoría
Descripción	El sistema debe permitir a los roles Manager y Admin crear nuevas categorías mediante POST /api/categories proporcionando nombre y descripción.
Actor	Manager, Admin
Precondición	El usuario debe estar autenticado con rol Manager o Admin.
Postcondición	Se crea una nueva categoría con atributo active en verdadero.
Prioridad	Alta
Estado	Implementado

RF-014 — Actualizar categoría

Identificador	RF-014
Nombre	Actualizar categoría
Descripción	El sistema debe permitir a los roles Manager y Admin actualizar los atributos de una categoría existente mediante PUT /api/categories/:id.
Actor	Manager, Admin
Precondición	La categoría debe existir; el usuario debe ser Manager o Admin.
Postcondición	El documento de la categoría se actualiza con los valores provistos.
Prioridad	Alta
Estado	Implementado

RF-015 — Eliminar categoría (lógica y física)

Identificador	RF-015
Nombre	Eliminar categoría (lógica y física)
Descripción	El sistema debe permitir al rol Admin eliminar categorías mediante DELETE /api/categories/:id, con la misma semántica de doble invocación que el resto de los módulos.

Actor	Admin
Precondición	La categoría debe existir; el solicitante debe ser Admin.
Postcondición	La categoría queda inactiva o eliminada físicamente según el estado previo.
Prioridad	Alta
Estado	Implementado

3.4 Módulo de Gestión de Subcategorías

RF-016 — Listar y consultar subcategorías públicamente

Identificador	RF-016
Nombre	Listar y consultar subcategorías públicamente
Descripción	El sistema debe exponer la lista de subcategorías activas y permitir consultar el detalle de cada una mediante GET /api/subcategories y GET /api/subcategories/:id sin requerir autenticación.
Actor	Cualquier usuario
Precondición	Ninguna.
Postcondición	Se retornan las subcategorías activas con su categoría padre referenciada.
Prioridad	Alta
Estado	Implementado

RF-017 — Crear subcategoría

Identificador	RF-017
Nombre	Crear subcategoría
Descripción	El sistema debe permitir a Manager y Admin crear subcategorías mediante POST /api/subcategories indicando el identificador de la categoría padre.
Actor	Manager, Admin
Precondición	La categoría padre debe existir y estar activa; el usuario debe ser Manager o Admin.
Postcondición	Se crea la subcategoría asociada a su categoría padre.
Prioridad	Alta
Estado	Implementado

RF-018 — Actualizar subcategoría

Identificador	RF-018
Nombre	Actualizar subcategoría
Descripción	El sistema debe permitir a Manager y Admin modificar los atributos de una subcategoría existente mediante PUT /api/subcategories/:id, incluyendo la posibilidad de reasignar la categoría padre.
Actor	Manager, Admin
Precondición	La subcategoría debe existir; el usuario debe ser Manager o Admin.
Postcondición	El documento se actualiza con los valores provistos.
Prioridad	Media
Estado	Implementado

RF-019 — Eliminar subcategoría

Identificador	RF-019
Nombre	Eliminar subcategoría
Descripción	El sistema debe permitir al rol Admin eliminar subcategorías mediante DELETE /api/subcategories/:id, con la misma semántica de doble invocación lógica/física.
Actor	Admin
Precondición	La subcategoría debe existir; el solicitante debe ser Admin.
Postcondición	La subcategoría queda inactiva o eliminada físicamente según el estado previo.
Prioridad	Alta
Estado	Implementado

3.5 Módulo de Gestión de Productos**RF-020 — Crear producto con variantes**

Identificador	RF-020
Nombre	Crear producto con variantes
Descripción	El sistema debe permitir a Manager y Admin crear productos mediante POST /api/products. Cada producto debe contener al menos nombre, marca, categoría, subcategoría y un arreglo de variantes con los atributos talla,

	color, SKU, stock, precio de costo y precio de venta. El slug se genera automáticamente a partir del nombre.
Actor	Manager, Admin
Precondición	La categoría y subcategoría referenciadas deben existir y estar activas. El usuario debe ser Manager o Admin.
Postcondición	Se crea el producto con su slug único, sus variantes y un atributo active en verdadero. Si se proporcionan descuentos, se almacenan con su tipo (percentage o fixed), valor y vigencia.
Prioridad	Alta
Estado	Implementado

RF-021 — Actualizar producto

Identificador	RF-021
Nombre	Actualizar producto
Descripción	El sistema debe permitir a Manager y Admin actualizar cualquier atributo de un producto existente, incluyendo el arreglo de variantes y el arreglo de descuentos, mediante PUT /api/products/:id.
Actor	Manager, Admin
Precondición	El producto debe existir; el usuario debe ser Manager o Admin.
Postcondición	El producto se actualiza con los valores provistos. Si se modifica el nombre, el slug se regenera respetando la unicidad.
Prioridad	Alta
Estado	Implementado

RF-022 — Cálculo automático de precio final, utilidad y margen

Identificador	RF-022
Nombre	Cálculo automático de precio final, utilidad y margen
Descripción	El sistema debe calcular dinámicamente, mediante campos virtuales del modelo, el precio final aplicando descuentos vigentes, la utilidad (precio de venta menos precio de costo) y el margen porcentual de utilidad. El campo virtual isOnDiscount debe indicar si el producto tiene un descuento activo según las fechas de vigencia.
Actor	Sistema
Precondición	El producto contiene al menos una variante con precios de costo y venta válidos.

Postcondición	Las respuestas que incluyan el producto exponen los campos virtuales finalPrice, profit, profitMargin e isOnDiscount calculados al momento de la consulta.
Prioridad	Alta
Estado	Implementado

RF-023 — Eliminar producto

Identificador	RF-023
Nombre	Eliminar producto
Descripción	El sistema debe permitir al rol Admin eliminar productos mediante DELETE /api/products/:id, aplicando la semántica de eliminación lógica seguida de eliminación física.
Actor	Admin
Precondición	El producto debe existir; el solicitante debe ser Admin.
Postcondición	El producto queda inactivo o eliminado físicamente según el estado previo.
Prioridad	Alta
Estado	Implementado

RF-024 — Indexación full-text de productos

Identificador	RF-024
Nombre	Indexación full-text de productos
Descripción	El modelo de Producto debe declarar índices de texto sobre los campos relevantes (nombre, descripción, marca, etiquetas) para soportar búsquedas full-text eficientes a través del operador \$text de MongoDB.
Actor	Sistema (backend)
Precondición	Ninguna.
Postcondición	Las consultas con el parámetro de búsqueda q ejecutan una búsqueda full-text indexada.
Prioridad	Alta
Estado	Implementado

3.6 Módulo de Catálogo Público

RF-025 — Listar productos con filtros y paginación

Identificador	RF-025
Nombre	Listar productos con filtros y paginación
Descripción	El sistema debe permitir consultar productos mediante GET /api/products aceptando los parámetros opcionales q (búsqueda full-text), inStock (booleano), priceMin, priceMax, sortBy, page y limit. La respuesta debe incluir los productos paginados y los metadatos de paginación.
Actor	Cualquier usuario
Precondición	Ninguna.
Postcondición	Se retorna un subconjunto de productos activos que cumplen los filtros, ordenados según sortBy y paginados según page y limit.
Prioridad	Alta
Estado	Implementado

RF-026 — Consultar producto por slug

Identificador	RF-026
Nombre	Consultar producto por slug
Descripción	El sistema debe permitir consultar el detalle completo de un producto mediante GET /api/products/slug/:slug, devolviendo todas sus variantes, descuentos vigentes y campos virtuales.
Actor	Cualquier usuario
Precondición	El slug debe corresponder a un producto activo.
Postcondición	Se retorna el documento completo del producto.
Prioridad	Alta
Estado	Implementado

RF-027 — Filtrar productos por categoría

Identificador	RF-027
Nombre	Filtrar productos por categoría
Descripción	El sistema debe permitir consultar productos asociados a una categoría específica mediante GET /api/products/category/:categoryId.
Actor	Cualquier usuario
Precondición	La categoría debe existir.
Postcondición	Se retornan los productos activos de la categoría indicada.

Prioridad	Media
Estado	Implementado

RF-028 — Filtrar productos por subcategoría

Identificador	RF-028
Nombre	Filtrar productos por subcategoría
Descripción	El sistema debe permitir consultar productos asociados a una subcategoría específica mediante GET /api/products/subcategory/:subcategoryId.
Actor	Cualquier usuario
Precondición	La subcategoría debe existir.
Postcondición	Se retornan los productos activos de la subcategoría indicada.
Prioridad	Media
Estado	Implementado

RF-029 — Filtrar productos por marca

Identificador	RF-029
Nombre	Filtrar productos por marca
Descripción	El sistema debe permitir consultar productos por marca mediante GET /api/products/brand/:brand.
Actor	Cualquier usuario
Precondición	Ninguna.
Postcondición	Se retornan los productos activos cuya marca coincide con el parámetro.
Prioridad	Media
Estado	Implementado

RF-030 — Filtrar productos por etiqueta

Identificador	RF-030
Nombre	Filtrar productos por etiqueta
Descripción	El sistema debe permitir consultar productos por etiqueta mediante GET /api/products/tags/:tag.
Actor	Cualquier usuario

Precondición	Ninguna.
Postcondición	Se retornan los productos activos que incluyen la etiqueta solicitada en su arreglo tags.
Prioridad	Media
Estado	Implementado

3.7 Módulo de Perfil de Usuario

RF-031 — Consultar perfil propio

Identificador	RF-031
Nombre	Consultar perfil propio
Descripción	El sistema debe permitir a un usuario autenticado consultar los datos de su propio perfil mediante GET /api/users/profile.
Actor	Usuario autenticado
Precondición	El usuario debe poseer una sesión válida.
Postcondición	Se retorna el documento del usuario sin el campo password.
Prioridad	Alta
Estado	Implementado

RF-032 — Eliminar perfil propio

Identificador	RF-032
Nombre	Eliminar perfil propio
Descripción	El sistema debe permitir a un usuario autenticado solicitar la eliminación de su propia cuenta mediante DELETE /api/users/profile, ejecutando una eliminación lógica.
Actor	Usuario autenticado
Precondición	El usuario debe poseer una sesión válida.
Postcondición	El atributo active del usuario pasa a falso, la sesión se invalida en el cliente y el usuario queda redirigido al estado anónimo.
Prioridad	Alta
Estado	Implementado

RF-033 — Editar datos del perfil propio

Identificador	RF-033
Nombre	Editar datos del perfil propio
Descripción	El sistema debe ofrecer una interfaz dentro del componente MyProfilePanel que permita al usuario editar sus datos personales (nombre, correo, contraseña), invocando el endpoint correspondiente del backend.
Actor	Usuario autenticado
Precondición	El usuario debe poseer una sesión válida.
Postcondición	Los datos del perfil se actualizan en el backend y la vista del frontend refleja los nuevos valores.
Prioridad	Media
Estado	Parcial

3.8 Módulo de Panel de Administración

RF-034 — Acceso al panel administrativo segmentado por rol

Identificador	RF-034
Nombre	Acceso al panel administrativo segmentado por rol
Descripción	El frontend debe exponer rutas administrativas bajo /admin con políticas de acceso por rol: /admin/users es exclusiva del rol Admin, mientras que /admin/categories, /admin/subcategories y /admin/products son accesibles tanto por Admin como por Manager.
Actor	Admin, Manager
Precondición	El usuario debe estar autenticado y poseer el rol requerido.
Postcondición	Se renderizan los paneles CRUD correspondientes (UsersCrudPanel, CategoriesCrudPanel, SubcategoriesCrudPanel, ProductsCrudPanel).
Prioridad	Alta
Estado	Implementado

RF-035 — Operaciones CRUD desde la interfaz

Identificador	RF-035
Nombre	Operaciones CRUD desde la interfaz
Descripción	Cada panel administrativo debe ofrecer una interfaz visual coherente con tablas, formularios modales y acciones de creación, edición y eliminación que invoquen los endpoints del backend.

Actor	Admin, Manager
Precondición	Acceso autorizado al panel correspondiente.
Postcondición	Las operaciones se reflejan tanto en la base de datos como en la vista, con notificaciones visuales del resultado.
Prioridad	Alta
Estado	Implementado

RF-036 — Visualización del estado activo/inactivo

Identificador	RF-036
Nombre	Visualización del estado activo/inactivo
Descripción	Los paneles administrativos deben mostrar visualmente el estado activo o inactivo de cada entidad y permitir filtrar entre ellos cuando corresponda.
Actor	Admin, Manager
Precondición	Acceso autorizado al panel correspondiente.
Postcondición	El usuario puede distinguir registros activos de inactivos sin necesidad de inspección manual.
Prioridad	Media
Estado	Implementado

3.9 Módulo de Gestión de Direcciones

Cada usuario puede registrar múltiples direcciones de envío. Una de ellas puede marcarse como predeterminada y será propuesta automáticamente al confirmar un pedido. Las direcciones se asocian a la entidad address según el diagrama de clases (relación 1..N con users) y son referenciadas desde la entidad orders.

RF-037 — Crear dirección

Identificador	RF-037
Nombre	Crear dirección
Descripción	El sistema debe permitir a un usuario autenticado registrar una nueva dirección de envío proporcionando calle, número, ciudad y código postal. Si es la primera dirección del usuario, se marca automáticamente como predeterminada.
Actor	Client, Manager, Admin
Precondición	El usuario debe estar autenticado.

Postcondición	Se crea un registro en la entidad address asociado al user_id del solicitante. El campo is_default se establece en verdadero si era la primera dirección.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-038 — Listar direcciones propias

Identificador	RF-038
Nombre	Listar direcciones propias
Descripción	El sistema debe permitir a un usuario autenticado consultar todas sus direcciones registradas, con la dirección predeterminada claramente identificada.
Actor	Client, Manager, Admin (sobre las propias)
Precondición	El usuario debe estar autenticado.
Postcondición	Se retorna el arreglo de direcciones del usuario, incluyendo el indicador is_default.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-039 — Editar dirección propia

Identificador	RF-039
Nombre	Editar dirección propia
Descripción	El sistema debe permitir al usuario editar cualquiera de sus direcciones. Las modificaciones no afectan retroactivamente a pedidos ya emitidos con dicha dirección.
Actor	Propietario de la dirección
Precondición	La dirección debe pertenecer al usuario autenticado.
Postcondición	El registro de la dirección se actualiza con los nuevos valores. Los pedidos históricos conservan los datos vigentes al momento de su confirmación.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

RF-040 — Eliminar dirección propia

Identificador	RF-040
----------------------	--------

Nombre	Eliminar dirección propia
Descripción	El sistema debe permitir al usuario eliminar cualquiera de sus direcciones. Si la dirección eliminada era la predeterminada y existen otras, el sistema debe promover automáticamente la más antigua como nueva predeterminada.
Actor	Propietario de la dirección
Precondición	La dirección debe pertenecer al usuario y no estar referenciada por un pedido en estado activo distinto de DELIVERED o CANCELLED.
Postcondición	La dirección se elimina lógicamente. Si era la predeterminada, otra dirección del usuario asume ese rol.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

RF-041 — Marcar dirección como predeterminada

Identificador	RF-041
Nombre	Marcar dirección como predeterminada
Descripción	El sistema debe permitir al usuario designar cualquiera de sus direcciones como predeterminada mediante el método setDefault(). Sólo una dirección por usuario puede estar marcada como predeterminada simultáneamente.
Actor	Propietario de la dirección
Precondición	La dirección debe pertenecer al usuario autenticado.
Postcondición	La dirección seleccionada queda con is_default = true; cualquier dirección previamente predeterminada del mismo usuario queda con is_default = false.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

3.10 Módulo de Carrito de Compras

El carrito de compras se modela como una instancia de la entidad orders en estado CART. Cada usuario puede mantener a lo sumo un pedido en estado CART simultáneamente. Las operaciones de carrito manipulan los registros de la entidad order_item asociados a dicho pedido.

RF-042 — Agregar producto al carrito

Identificador	RF-042
Nombre	Agregar producto al carrito

Descripción	El sistema debe permitir a un usuario autenticado añadir un producto a su carrito indicando producto y cantidad. Si el usuario no tiene un pedido en estado CART, se crea uno automáticamente. Si el producto ya está en el carrito, se incrementa la cantidad existente.
Actor	Client, Manager, Admin
Precondición	El usuario debe estar autenticado, el producto debe estar activo y disponer de stock suficiente.
Postcondición	Existe (o se actualiza) un registro en order_item asociado al pedido en estado CART del usuario, con quantity actualizada y subtotal recalculado.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-043 — Consultar carrito

Identificador	RF-043
Nombre	Consultar carrito
Descripción	El sistema debe permitir al usuario consultar el contenido actual de su carrito, incluyendo lista de items con producto, cantidad, precio unitario y subtotal, así como el total agregado del pedido.
Actor	Client, Manager, Admin
Precondición	El usuario debe estar autenticado.
Postcondición	Se retorna el pedido en estado CART del usuario con todos sus order_item y el total calculado dinámicamente.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-044 — Modificar cantidad de un item del carrito

Identificador	RF-044
Nombre	Modificar cantidad de un item del carrito
Descripción	El sistema debe permitir al usuario actualizar la cantidad de un producto presente en su carrito mediante el método updateQuantity() de order_item. Si la nueva cantidad es cero, el item se elimina del carrito.
Actor	Client, Manager, Admin
Precondición	El item debe pertenecer al carrito del usuario y la nueva cantidad no debe exceder el stock disponible.

Postcondición	El order_item se actualiza con la nueva cantidad y subtotal recalculado, o se elimina si la cantidad es cero. El total del pedido se recalcula.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-045 — Eliminar ítem del carrito

Identificador	RF-045
Nombre	Eliminar ítem del carrito
Descripción	El sistema debe permitir al usuario remover un producto específico de su carrito.
Actor	Client, Manager, Admin
Precondición	El ítem debe pertenecer al carrito del usuario.
Postcondición	El order_item se elimina y el total del pedido se recalcula.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-046 — Vaciar carrito

Identificador	RF-046
Nombre	Vaciar carrito
Descripción	El sistema debe permitir al usuario eliminar todos los ítems de su carrito en una sola operación.
Actor	Client, Manager, Admin
Precondición	El usuario debe tener un pedido en estado CART.
Postcondición	Todos los order_item asociados al carrito se eliminan. El pedido en estado CART permanece con total cero o se elimina, según política definida.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

3.11 Módulo de Pedidos (Órdenes)

Una vez confirmado el carrito, el pedido transita por un ciclo de vida explícito de estados: CART → PENDING → CONFIRMED → SHIPPED → DELIVERED, con la posibilidad de transición a

CANCELLED desde los estados previos a SHIPPED. Cada transición es ejecutada por un actor autorizado y dispara efectos colaterales en pagos, envíos y facturación.

RF-047 — Confirmar pedido (checkout)

Identificador	RF-047
Nombre	Confirmar pedido (checkout)
Descripción	El sistema debe permitir al usuario confirmar su carrito convirtiéndolo en un pedido formal. La operación requiere seleccionar una dirección de envío válida (<code>address_id</code>) y produce la transición de estado <code>CART</code> → <code>PENDING</code> . El sistema valida la disponibilidad de stock de cada ítem y reserva las unidades correspondientes.
Actor	Propietario del carrito
Precondición	El carrito debe contener al menos un ítem; todos los productos deben tener stock suficiente; el usuario debe tener al menos una dirección registrada.
Postcondición	El pedido transita a estado <code>PENDING</code> , se asigna <code>address_id</code> , se establece <code>order_date</code> , se calcula y persiste el total mediante <code>calculateTotal()</code> , y el stock de cada producto se decrementa atómicamente.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-048 — Cálculo automático del total del pedido

Identificador	RF-048
Nombre	Cálculo automático del total del pedido
Descripción	El sistema debe exponer el método <code>calculateTotal()</code> en la entidad <code>orders</code> para calcular el total a partir de la sumatoria de los subtotales de sus <code>order_item</code> . Este método se invoca al modificar el carrito y al confirmar el pedido.
Actor	Sistema
Precondición	El pedido contiene al menos un <code>order_item</code> .
Postcondición	El campo total del pedido refleja la suma exacta de los subtotales de sus ítems con precisión <code>DECIMAL(12,2)</code> .
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-049 — Cancelar pedido

Identificador	RF-049
----------------------	--------

Nombre	Cancelar pedido
Descripción	El sistema debe permitir cancelar un pedido cuyo estado actual sea PENDING o CONFIRMED. La cancelación restituye el stock reservado y, si existía un pago COMPLETED, marca un reembolso pendiente. No es posible cancelar pedidos en estado SHIPPED, DELIVERED o ya CANCELLED.
Actor	Propietario del pedido (PENDING) / Admin (cualquiera previo a SHIPPED)
Precondición	El pedido debe existir y estar en estado PENDING o CONFIRMED.
Postcondición	El estado del pedido transita a CANCELLED, el stock de cada producto se incrementa para restituir las unidades reservadas, y los pagos asociados se marcan para reembolso si aplica.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-050 — Listar pedidos propios

Identificador	RF-050
Nombre	Listar pedidos propios
Descripción	El sistema debe permitir a un usuario consultar el historial de sus pedidos en cualquier estado distinto de CART, con los datos resumidos (fecha, estado, total).
Actor	Client, Manager, Admin (sobre los propios)
Precondición	El usuario debe estar autenticado.
Postcondición	Se retorna el arreglo de pedidos del usuario ordenado por order_date descendente.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-051 — Listar todos los pedidos (administración)

Identificador	RF-051
Nombre	Listar todos los pedidos (administración)
Descripción	El sistema debe permitir a los roles Admin y Manager consultar todos los pedidos del sistema, con filtros por estado, rango de fechas y usuario. La consulta soporta paginación.
Actor	Admin, Manager
Precondición	El usuario debe estar autenticado con rol Admin o Manager.

Postcondición	Se retorna la lista paginada de pedidos según los filtros provistos.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-052 — Actualizar estado del pedido

Identificador	RF-052
Nombre	Actualizar estado del pedido
Descripción	El sistema debe permitir a Admin y Manager invocar el método <code>updateStatus()</code> para promover el pedido a través de los estados <code>CONFIRMED</code> → <code>SHIPPED</code> → <code>DELIVERED</code> . Cada transición valida la coherencia con los estados de pago y envío.
Actor	Admin, Manager
Precondición	El pedido debe estar en un estado válido para la transición solicitada (p. ej. <code>CONFIRMED</code> para pasar a <code>SHIPPED</code> , con un envío asociado en estado <code>SHIPPED</code>).
Postcondición	El estado del pedido se actualiza, el campo <code>updated_at</code> se refresca y se notifica el cambio a los módulos dependientes (envío, facturación).
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-053 — Consultar detalle de un pedido

Identificador	RF-053
Nombre	Consultar detalle de un pedido
Descripción	El sistema debe permitir consultar el detalle completo de un pedido, incluyendo sus <code>order_item</code> , dirección de envío, pago asociado, envío asociado y factura cuando exista.
Actor	Propietario del pedido / Admin / Manager
Precondición	El pedido debe existir y el solicitante debe ser propietario o tener rol Admin o Manager.
Postcondición	Se retorna el documento del pedido con todas sus relaciones expandidas.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

3.12 Módulo de Procesamiento de Pagos

La entidad payment registra los pagos asociados a cada pedido. Cada pago atraviesa los estados PENDING, COMPLETED, FAILED o REFUNDED. La integración concreta con una pasarela de pagos externa será objeto de una especificación complementaria; este SRS define el contrato funcional del módulo.

RF-054 — Iniciar proceso de pago

Identificador	RF-054
Nombre	Iniciar proceso de pago
Descripción	El sistema debe permitir al propietario de un pedido iniciar un proceso de pago indicando el método (p. ej. tarjeta, transferencia). Se crea un registro en payment en estado PENDING con el monto igual al total del pedido.
Actor	Propietario del pedido
Precondición	El pedido debe estar en estado PENDING. No debe existir un pago previo en estado COMPLETED para el mismo pedido.
Postcondición	Se crea un registro de payment con status = PENDING, amount = orders.total, method y created_at. Se invoca processPayment() para delegar a la pasarela.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-055 — Procesar resultado del pago

Identificador	RF-055
Nombre	Procesar resultado del pago
Descripción	El método processPayment() debe interactuar con la pasarela externa y, según el resultado, actualizar el estado del pago a COMPLETED (con paid_at) o FAILED.
Actor	Sistema
Precondición	Existe un registro de payment en estado PENDING.
Postcondición	El estado del payment se actualiza. Si COMPLETED, se invoca la generación automática de factura (RF-066) y se promueve el pedido a CONFIRMED. Si FAILED, el pedido permanece en PENDING y el usuario puede reintentar.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-056 — Validar coherencia entre pago y total del pedido

Identificador	RF-056
Nombre	Validar coherencia entre pago y total del pedido
Descripción	El sistema debe verificar que el campo amount del payment coincida exactamente con el total del pedido al momento de iniciar el procesamiento. Cualquier divergencia debe rechazar la operación.
Actor	Sistema
Precondición	Existe un payment vinculado al pedido.
Postcondición	Si payment.amount $\neq$ orders.total, el pago se rechaza con código de error específico y el registro se marca como FAILED.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-057 — Solicitar reembolso

Identificador	RF-057
Nombre	Solicitar reembolso
Descripción	El sistema debe permitir al rol Admin invocar el método refund() sobre un pago en estado COMPLETED para iniciar un reembolso. La operación cambia el estado del pago a REFUNDED y, si el pedido aún no ha sido enviado, lo marca como CANCELLED.
Actor	Admin
Precondición	El pago debe estar en estado COMPLETED y el pedido no debe encontrarse en estado DELIVERED.
Postcondición	El estado del payment transita a REFUNDED. El pedido se actualiza a CANCELLED si corresponde y el stock se restituye.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

RF-058 — Consultar historial de pagos

Identificador	RF-058
Nombre	Consultar historial de pagos
Descripción	El sistema debe permitir al rol Admin consultar el historial completo de pagos del sistema, con filtros por estado, método, rango de fechas y usuario.
Actor	Admin
Precondición	El usuario debe estar autenticado con rol Admin.

Postcondición	Se retorna la lista paginada de pagos según los filtros provistos.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

RF-059 — Consultar pago de pedido propio

Identificador	RF-059
Nombre	Consultar pago de pedido propio
Descripción	El propietario de un pedido debe poder consultar el estado del pago asociado a sus propios pedidos.
Actor	Propietario del pedido
Precondición	El pedido debe pertenecer al usuario y debe existir un payment vinculado.
Postcondición	Se retorna el documento del payment con su estado y método.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

3.13 Módulo de Gestión de Envíos

La entidad shipment registra el envío físico de un pedido. Un pedido tiene a lo sumo un envío asociado. El envío atraviesa los estados PENDING, SHIPPED, IN_TRANSIT, DELIVERED y, eventualmente, RETURNED. La integración con operadores logísticos concretos queda fuera del alcance de este SRS.

RF-060 — Crear envío para un pedido

Identificador	RF-060
Nombre	Crear envío para un pedido
Descripción	El sistema debe permitir a Admin y Manager crear un registro de shipment para un pedido en estado CONFIRMED, especificando el método de envío (shipping_method). El envío inicia en estado PENDING.
Actor	Admin, Manager
Precondición	El pedido debe estar en estado CONFIRMED y no debe existir un envío previo asociado.
Postcondición	Se crea el registro shipment con status = PENDING, order_id, shipping_method y created_at. El pedido permanece en CONFIRMED.
Prioridad	Alta

Estado	Pendiente — Especificado en diagrama de clases
---------------	--

RF-061 — Asignar número de seguimiento

Identificador	RF-061
Nombre	Asignar número de seguimiento
Descripción	El sistema debe permitir a Admin y Manager invocar el método <code>setTrackingNumber()</code> para asignar el <code>tracking_number</code> provisto por el operador logístico al envío.
Actor	Admin, Manager
Precondición	El envío debe existir y estar en estado PENDING o SHIPPED.
Postcondición	El campo <code>tracking_number</code> se actualiza en el registro de shipment.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-062 — Actualizar estado del envío

Identificador	RF-062
Nombre	Actualizar estado del envío
Descripción	El sistema debe permitir a Admin y Manager invocar <code>updateStatus()</code> para promover el envío entre los estados PENDING → SHIPPED → IN_TRANSIT → DELIVERED. La transición a SHIPPED registra <code>shipped_at</code> y promueve el pedido a SHIPPED. La transición a DELIVERED registra <code>delivered_at</code> y promueve el pedido a DELIVERED.
Actor	Admin, Manager
Precondición	El envío debe estar en un estado válido para la transición solicitada.
Postcondición	El estado del shipment se actualiza, los timestamps correspondientes se registran y el estado del pedido asociado se promueve coherentemente.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-063 — Registrar devolución de envío

Identificador	RF-063
Nombre	Registrar devolución de envío

Descripción	El sistema debe permitir a Admin y Manager marcar un envío como RETURNED cuando el paquete sea devuelto por cualquier causa (rechazo del destinatario, dirección incorrecta, etc.).
Actor	Admin, Manager
Precondición	El envío debe estar en estado SHIPPED, IN_TRANSIT o DELIVERED.
Postcondición	El estado del shipment transita a RETURNED. Se registra el evento para análisis posterior y se inicia la política de reembolso si aplica.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

RF-064 — Consultar tracking de envío propio

Identificador	RF-064
Nombre	Consultar tracking de envío propio
Descripción	El propietario de un pedido debe poder consultar el estado y número de seguimiento del envío asociado a su pedido.
Actor	Propietario del pedido
Precondición	El pedido debe pertenecer al usuario y debe tener un shipment asociado.
Postcondición	Se retorna el documento del shipment con tracking_number, status y timestamps.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-065 — Listar envíos pendientes (operativa logística)

Identificador	RF-065
Nombre	Listar envíos pendientes (operativa logística)
Descripción	El sistema debe permitir a Admin y Manager listar los envíos en estado PENDING o SHIPPED para apoyar la operación logística diaria.
Actor	Admin, Manager
Precondición	El usuario debe estar autenticado con rol Admin o Manager.
Postcondición	Se retorna la lista de envíos filtrada por estado, ordenada por created_at ascendente.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

3.14 Módulo de Facturación

La entidad bill representa la factura emitida por un pedido cuyo pago haya sido completado. Cada factura tiene una referencia única (reference) y un total que coincide con el total del pedido al momento de la facturación. La emisión es un acto automático del sistema.

RF-066 — Generar factura automáticamente

Identificador	RF-066
Nombre	Generar factura automáticamente
Descripción	Al transicionar un payment al estado COMPLETED, el sistema debe generar automáticamente un registro en la entidad bill con order_id, total y created_at. La operación es idempotente: si ya existe una factura para el pedido, no se genera una nueva.
Actor	Sistema
Precondición	Existe un payment en estado COMPLETED asociado al pedido y no existe una bill previa para ese pedido.
Postcondición	Se crea el registro bill con la referencia generada por generateReference() y el total tomado de orders.total.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-067 — Generar referencia única de factura

Identificador	RF-067
Nombre	Generar referencia única de factura
Descripción	El método generateReference() de la entidad bill debe producir una cadena alfanumérica única en el sistema que sirva como identificador legible de la factura. La unicidad se garantiza por la restricción «UK» del campo reference.
Actor	Sistema
Precondición	Se está creando una nueva factura.
Postcondición	El campo reference contiene una cadena única conforme a la política de generación definida (p. ej. INV-AAAA-NNNNNN).
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-068 — Consultar factura propia

Identificador	RF-068
Nombre	Consultar factura propia
Descripción	El propietario del pedido debe poder consultar la factura asociada a sus pedidos pagados.
Actor	Propietario del pedido
Precondición	El pedido debe pertenecer al usuario y debe existir una bill asociada.
Postcondición	Se retorna el documento de la factura con todos sus campos.
Prioridad	Alta
Estado	Pendiente — Especificado en diagrama de clases

RF-069 — Listar facturas (administración)

Identificador	RF-069
Nombre	Listar facturas (administración)
Descripción	El sistema debe permitir a los roles Admin y Manager consultar todas las facturas emitidas, con filtros por rango de fechas y referencia.
Actor	Admin, Manager
Precondición	El usuario debe estar autenticado con rol Admin o Manager.
Postcondición	Se retorna la lista paginada de facturas según los filtros provistos.
Prioridad	Media
Estado	Pendiente — Especificado en diagrama de clases

4. Requerimientos No Funcionales

4.1 Seguridad

RNF-001 — Hashing de contraseñas con bcrypt

Identificador	RNF-001
Nombre	Hashing de contraseñas con bcrypt
Categoría	Seguridad
Descripción	Todas las contraseñas deben almacenarse exclusivamente como hash bcrypt con un factor de costo no inferior a 10. La contraseña en claro nunca debe persistirse en la base de datos, en logs, ni retornarse en respuestas de la API.
Criterio de aceptación	Inspección del modelo User: el campo password debe ser excluido por toJSON(). Pruebas de penetración deben confirmar la imposibilidad de recuperar contraseñas en claro mediante respuestas de la API.
Estado	Implementado

RNF-002 — Autenticación basada en JWT

Identificador	RNF-002
Nombre	Autenticación basada en JWT
Categoría	Seguridad
Descripción	El sistema debe emplear tokens JWT firmados con un secreto seguro almacenado en variables de entorno. Los tokens deben tener una vigencia máxima de 30 días y transportarse en la cabecera Authorization bajo el esquema Bearer.
Criterio de aceptación	Verificación automatizada: cualquier solicitud privada sin token o con token expirado debe responder con código HTTP 401.
Estado	Implementado

RNF-003 — Validación de entradas

Identificador	RNF-003
Nombre	Validación de entradas
Categoría	Seguridad
Descripción	Todas las rutas de escritura deben aplicar validaciones explícitas mediante express-validator antes de invocar al controlador, incluyendo formato de correo, longitud mínima de contraseña, tipos numéricos y existencia de referencias a otras entidades.

Criterio de aceptación	Las solicitudes con cuerpos malformados o campos inválidos deben responder con código HTTP 400 y un mensaje descriptivo. Cobertura de validación medible al 100% sobre rutas de escritura.
Estado	Implementado

RNF-004 — Control de acceso por rol

Identificador	RNF-004
Nombre	Control de acceso por rol
Categoría	Seguridad
Descripción	Cada ruta privada debe verificar el rol del usuario autenticado contra una política de roles permitidos. Los intentos de acceso por roles no autorizados deben ser rechazados sin exponer información del recurso.
Criterio de aceptación	Pruebas de regresión: para cada ruta restringida, una solicitud con un rol no autorizado debe responder con código HTTP 403.
Estado	Implementado

RNF-005 — Eliminación lógica como mecanismo por defecto

Identificador	RNF-005
Nombre	Eliminación lógica como mecanismo por defecto
Categoría	Seguridad / Integridad
Descripción	Todas las operaciones DELETE deben ejecutar primero una eliminación lógica (active = false). La eliminación física sólo procede en una segunda invocación explícita y exclusivamente por el rol Admin.
Criterio de aceptación	Inspección de los controladores: ningún controlador permite eliminación física en una sola llamada. Pruebas de auditoría confirman la trazabilidad de los registros inactivos.
Estado	Implementado

RNF-006 — Configuración de CORS

Identificador	RNF-006
Nombre	Configuración de CORS
Categoría	Seguridad
Descripción	El backend debe configurar políticas CORS explícitas que restrinjan los orígenes permitidos según el entorno (desarrollo, staging, producción).

Criterio de aceptación	Inspección del middleware CORS: la lista de orígenes permitidos debe estar definida y no contener comodines en producción.
Estado	Implementado

4.2 Rendimiento

RNF-007 — Tiempo de respuesta de la API

Identificador	RNF-007
Nombre	Tiempo de respuesta de la API
Categoría	Rendimiento
Descripción	Las consultas estándar al catálogo (GET /api/products con filtros y paginación de hasta 24 elementos por página) deben responder en menos de 500 milisegundos en el percentil 95 bajo condiciones nominales de carga.
Criterio de aceptación	Pruebas de carga con herramientas como k6 o Apache JMeter: P95 < 500 ms con 50 usuarios concurrentes y un dataset de 10.000 productos.
Estado	Implementado

RNF-008 — Paginación obligatoria en listados extensos

Identificador	RNF-008
Nombre	Paginación obligatoria en listados extensos
Categoría	Rendimiento
Descripción	Las consultas a colecciones potencialmente grandes (productos, usuarios) deben soportar y aplicar paginación por defecto mediante los parámetros page y limit, con un valor por defecto razonable (por ejemplo, 12 o 20 elementos).
Criterio de aceptación	Inspección del controlador de productos: ausencia de invocaciones a find() sin .limit() ni .skip().
Estado	Implementado

RNF-009 — Índices en MongoDB

Identificador	RNF-009
Nombre	Índices en MongoDB
Categoría	Rendimiento

Descripción	Los campos utilizados frecuentemente como criterios de búsqueda u ordenamiento (slug, category, subcategory, brand, tags, índice de texto) deben estar respaldados por índices declarados en los esquemas de Mongoose.
Criterio de aceptación	Verificación: db.products.getIndexes() debe incluir los índices declarados; los planes de consulta (explain()) deben mostrar uso de índice y no COLLSCAN.
Estado	Implementado

4.3 Usabilidad

RNF-010 — Diseño responsivo

Identificador	RNF-010
Nombre	Diseño responsivo
Categoría	Usabilidad
Descripción	Toda la interfaz frontend debe adaptarse correctamente a resoluciones desde 360 px (móviles) hasta 1920 px (escritorio amplio), conservando la legibilidad y la accesibilidad de los controles principales.
Criterio de aceptación	Validación visual en breakpoints definidos (móvil, tablet, escritorio). Ningún elemento crítico debe quedar fuera del viewport ni superponerse.
Estado	Implementado

RNF-011 — Animaciones e interacciones fluidas

Identificador	RNF-011
Nombre	Animaciones e interacciones fluidas
Categoría	Usabilidad
Descripción	El sistema utiliza Framer Motion, GSAP y Lenis para animaciones e interpolaciones de scroll. Las animaciones no deben interferir con la usabilidad ni reducir la tasa de fotogramas por debajo de 50 FPS en hardware de gama media.
Criterio de aceptación	Auditoría con Chrome DevTools Performance: tasa de fotogramas estable y sin long tasks superiores a 200 ms durante interacciones típicas.
Estado	Implementado

RNF-012 — Sistema de tokens de diseño

Identificador	RNF-012
----------------------	---------

Nombre	Sistema de tokens de diseño
Categoría	Usabilidad / Mantenibilidad
Descripción	Todos los colores, espaciados y duraciones de animación deben provenir de variables CSS declaradas como tokens (--kn-*, --sp-*, --dur-*) y no estar codificados directamente en componentes individuales.
Criterio de aceptación	Inspección de los archivos CSS: ausencia de valores hexadecimales o numéricos crudos fuera del archivo de tokens.
Estado	Implementado

RNF-013 — Mensajes de error claros

Identificador	RNF-013
Nombre	Mensajes de error claros
Categoría	Usabilidad
Descripción	Las respuestas de error de la API deben incluir un mensaje legible que el frontend pueda exponer al usuario en su idioma, sin filtrar detalles internos del sistema (stacktraces, nombres de tablas, etc.).
Criterio de aceptación	Inspección de respuestas 4xx/5xx: ningún mensaje contiene información sensible. Las pruebas de QA validan la legibilidad de los mensajes en flujos típicos de error.
Estado	Implementado

4.4 Mantenibilidad

RNF-014 — Separación estricta de capas

Identificador	RNF-014
Nombre	Separación estricta de capas
Categoría	Mantenibilidad
Descripción	El backend debe respetar la separación entre rutas, middlewares de validación, controladores y modelos. Ningún controlador debe contener lógica de validación duplicada que ya exista en un middleware, ni acceso directo a la base de datos sin pasar por el modelo de Mongoose.
Criterio de aceptación	Revisión de código: cada nuevo módulo debe seguir la estructura Route → Middleware → Controller → Model. Auditorías periódicas confirman la separación.
Estado	Implementado

RNF-015 — Convenciones de nomenclatura

Identificador	RNF-015
Nombre	Convenciones de nomenclatura
Categoría	Mantenibilidad
Descripción	El código debe respetar convenciones uniformes: camelCase para variables y funciones, PascalCase para componentes React y modelos de Mongoose, kebab-case para nombres de archivos no componentes.
Criterio de aceptación	Configuración de un linter (ESLint) con reglas de nomenclatura. Revisiones de pull request bloquean introducciones de nomenclatura inconsistente.
Estado	Implementado

RNF-016 — Documentación interna actualizada

Identificador	RNF-016
Nombre	Documentación interna actualizada
Categoría	Mantenibilidad
Descripción	Los cambios estructurales en el sistema deben reflejarse en la documentación interna ubicada en Docs/01-ANALYSIS y Docs/02-DESING (casos de uso, diagrama de clases, diagrama de despliegue, BPMN).
Criterio de aceptación	Revisión por sprint: la documentación debe estar sincronizada con la versión actual del código en cada release mayor.
Estado	Implementado

4.5 Compatibilidad**RNF-017 — Compatibilidad con navegadores modernos**

Identificador	RNF-017
Nombre	Compatibilidad con navegadores modernos
Categoría	Compatibilidad
Descripción	El frontend debe operar correctamente en las dos últimas versiones mayores de Chrome, Firefox, Edge y Safari.
Criterio de aceptación	Pruebas manuales y automatizadas de regresión visual y funcional en los navegadores indicados antes de cada release.
Estado	Implementado

RNF-018 — Versión de Node.js soportada

Identificador	RNF-018
Nombre	Versión de Node.js soportada
Categoría	Compatibilidad
Descripción	El backend debe ejecutarse correctamente sobre versiones LTS activas de Node.js (20.x o superior).
Criterio de aceptación	El package.json declara explícitamente engines.node. La integración continua valida la ejecución sobre la versión LTS objetivo.
Estado	Implementado

4.6 Disponibilidad**RNF-019 — Disponibilidad operativa**

Identificador	RNF-019
Nombre	Disponibilidad operativa
Categoría	Disponibilidad
Descripción	En entorno de producción, el sistema debe alcanzar una disponibilidad mensual no inferior al 99,5%. En entornos de desarrollo y staging no se exige un SLA, pero el servicio debe ser reinicializable mediante un único comando documentado.
Criterio de aceptación	Monitoreo continuo del endpoint de salud con herramientas de uptime. Reportes mensuales de disponibilidad acumulada.
Estado	Implementado

RNF-020 — Manejo robusto de errores

Identificador	RNF-020
Nombre	Manejo robusto de errores
Categoría	Disponibilidad / Confiabilidad
Descripción	El backend debe interceptar errores no controlados mediante un middleware de manejo de errores que evite la caída del proceso y registre el incidente para revisión.
Criterio de aceptación	Pruebas de inyección de errores: ninguna entrada inválida o excepción interna debe terminar el proceso de Node.js.
Estado	Implementado

4.7 Escalabilidad

RNF-021 — Arquitectura sin estado en el backend

Identificador	RNF-021
Nombre	Arquitectura sin estado en el backend
Categoría	Escalabilidad
Descripción	El backend no debe almacenar estado de sesión en memoria; toda la información de sesión se transporta en el JWT. Esto habilita el escalado horizontal mediante múltiples instancias detrás de un balanceador.
Criterio de aceptación	Inspección del código: ausencia de variables globales que retengan estado de usuarios entre solicitudes.
Estado	Implementado

RNF-022 — Capacidad de crecimiento del catálogo

Identificador	RNF-022
Nombre	Capacidad de crecimiento del catálogo
Categoría	Escalabilidad
Descripción	El sistema debe operar correctamente con catálogos de hasta 100.000 productos sin degradación significativa de tiempos de respuesta, gracias a los índices y a la paginación obligatoria.
Criterio de aceptación	Pruebas con dataset sintético de 100.000 productos: P95 de listado paginado se mantiene por debajo de 700 ms.
Estado	Implementado

4.8 Integridad transaccional

RNF-023 — Atomicidad del checkout

Identificador	RNF-023
Nombre	Atomicidad del checkout
Categoría	Integridad / Confiabilidad
Descripción	La confirmación de un pedido (transición CART → PENDING) debe ejecutarse como una operación atómica que comprenda la validación de stock, el decremento de stock de cada producto y el cambio de estado del pedido. Ante cualquier fallo intermedio, todos los efectos deben revertirse.

Criterio de aceptación	Pruebas de fallo inducido (kill -9 en mitad de la operación, falla de red simulada): tras la recuperación, el sistema queda en un estado consistente sin stock decrementado para pedidos no confirmados.
Estado	Pendiente — Especificado en diagrama de clases

RNF-024 — Concurrencia en operaciones de stock

Identificador	RNF-024
Nombre	Concurrencia en operaciones de stock
Categoría	Integridad
Descripción	El sistema debe manejar correctamente el escenario en que múltiples usuarios intenten comprar el mismo producto simultáneamente cuando el stock disponible sea inferior a la demanda agregada. Sólo deben confirmarse los pedidos que efectivamente disponen de stock.
Criterio de aceptación	Pruebas de concurrencia: con 50 solicitudes simultáneas sobre un producto con 10 unidades de stock, exactamente 10 pedidos deben confirmarse y los restantes 40 deben rechazarse con error claro.
Estado	Pendiente — Especificado en diagrama de clases

RNF-025 — Auditoría de pagos y pedidos

Identificador	RNF-025
Nombre	Auditoría de pagos y pedidos
Categoría	Trazabilidad / Cumplimiento
Descripción	Toda operación que modifique el estado de un pedido, pago, envío o factura debe quedar registrada con marca temporal, identificador del actor y estado previo y posterior. Estos registros deben ser inmutables una vez creados.
Criterio de aceptación	Inspección del log de auditoría: 100% de las transiciones de estado de pedidos y pagos producen un registro de auditoría con los campos requeridos.
Estado	Pendiente — Especificado en diagrama de clases

RNF-026 — Precisión monetaria

Identificador	RNF-026
Nombre	Precisión monetaria
Categoría	Integridad

Descripción	Todos los cálculos monetarios (subtotales, totales, montos de pago, totales de factura) deben utilizar tipos numéricos de precisión fija DECIMAL(10,2) o DECIMAL(12,2) según corresponda al diagrama de clases. No se admite el uso de tipos de coma flotante para valores monetarios.
Criterio de aceptación	Inspección de los esquemas de la base de datos: todos los campos monetarios usan DECIMAL con la precisión documentada. Pruebas funcionales validan la ausencia de errores de redondeo en sumatorias largas.
Estado	Pendiente — Especificado en diagrama de clases

5. Requerimientos de Interfaz

5.1 Interfaces de usuario

La interfaz de usuario está construida con Next.js 15 (App Router) y React 19. Las páginas se distribuyen entre rutas públicas, privadas y administrativas, con políticas de acceso aplicadas tanto en el frontend (mediante AuthContext) como en el backend (mediante middlewares de autorización).

Ruta	Acceso	Componente principal
/	Público	Hero + ProductCatalog (catálogo con filtros y paginación)
/auth	Público	AuthShellCreative (formularios de login y registro)
/profile	Privado (Client+)	MyProfilePanel
/admin/users	Solo Admin	UsersCrudPanel
/admin/categories	Admin / Manager	CategoriesCrudPanel
/admin/subcategories	Admin / Manager	SubcategoriesCrudPanel
/admin/products	Admin / Manager	ProductsCrudPanel

La gestión del estado de autenticación se concentra en AuthContext (React Context), que expone los valores user, loading, error, y las funciones login(), register() y logout(), además del booleano isAuthenticated. La sesión se persiste en localStorage.

5.2 Interfaces de hardware

El sistema no requiere interfaces de hardware específicas. Opera sobre hardware estándar de servidor y dispositivos cliente convencionales (computadoras, tabletas y teléfonos móviles) con conexión a Internet.

5.3 Interfaces de software

El sistema interactúa con los siguientes componentes de software:

- API REST del backend, accesible bajo el prefijo /api y servida por Express 5 sobre el puerto 3000.
- Base de datos MongoDB, accedida mediante Mongoose 9 con esquemas declarativos.
- Rewrites de Next.js que canalizan las solicitudes del frontend hacia el backend evitando errores de CORS en desarrollo.

El formato de comunicación entre frontend y backend es JSON con la siguiente estructura estándar:

```
{ "success": true, "data": { ... } }
```



```
{ "success": false, "message": "descripción del error" }
```

5.4 Interfaces de comunicación

- Protocolo HTTP/HTTPS sobre TCP/IP.
- Codificación de cuerpos de solicitud y respuesta en JSON (application/json).
- Autenticación mediante cabecera Authorization: Bearer <token>.
- Cabeceras CORS configuradas explícitamente para entornos de desarrollo y producción.

6. Restricciones y Criterios de Calidad

6.1 Reglas de negocio críticas

- RN-01: Sólo el rol Admin puede ejecutar eliminaciones físicas. Cualquier intento por otro rol debe ser rechazado con código HTTP 403.
- RN-02: El registro público (POST /api/users/register) sólo crea usuarios con rol Client. La asignación de roles Manager o Admin sólo es posible desde el panel administrativo por un usuario autenticado con rol Admin.
- RN-03: Las contraseñas nunca se persisten ni se transmiten en claro. El método toJSON() del modelo User excluye obligatoriamente el campo password.
- RN-04: La eliminación lógica (active = false) es la operación por defecto en todas las entidades. Los listados públicos (GET) deben filtrar siempre por active = true.
- RN-05: Un descuento aplica únicamente cuando la fecha actual está comprendida entre startDate y endDate. Los campos virtuales finalPrice y isOnDiscount deben respetar estrictamente esta condición.
- RN-06: El slug de un producto es único en el sistema. Si una actualización de nombre genera colisión de slug, el sistema debe resolver el conflicto añadiendo un sufijo o rechazando la operación con un mensaje claro.
- RN-07: Una subcategoría está obligatoriamente asociada a una categoría existente. La eliminación física de una categoría con subcategorías activas debe estar sujeta a una política explícita (rechazo o cascada documentada).
- RN-08: Un usuario puede tener a lo sumo un pedido en estado CART simultáneamente. Cualquier nueva adición al carrito debe operar sobre dicho pedido o crearlo si no existe.
- RN-09: El total de un pedido (orders.total) debe coincidir exactamente con la sumatoria de los subtotales de sus order_item. Cualquier divergencia detectada debe rechazar la operación de checkout o pago.
- RN-10: El stock de un producto se decrementa atómicamente al confirmar el pedido (CART → PENDING) y se restituye al cancelar el pedido. No se permite confirmar pedidos sin stock suficiente.
- RN-11: Un pedido en estado SHIPPED, DELIVERED o CANCELLED no puede ser cancelado. Las devoluciones desde SHIPPED, IN_TRANSIT o DELIVERED se gestionan a través del estado RETURNED del envío.
- RN-12: La generación de factura es un acto automático del sistema disparado por la transición de un payment a estado COMPLETED. La operación es idempotente: nunca se emite más de una factura por pedido.
- RN-13: La referencia de factura (bill.reference) debe ser única en todo el sistema. La unicidad se garantiza por restricción de base de datos.
- RN-14: Sólo el rol Admin puede iniciar reembolsos. Un reembolso sólo procede si el pago está en estado COMPLETED y el pedido no se encuentra en estado DELIVERED.
- RN-15: La dirección registrada en un pedido (orders.address_id) refleja los datos vigentes al momento de la confirmación. Modificaciones posteriores de la dirección por parte del usuario no afectan retroactivamente a pedidos ya emitidos.

6.2 Criterios de aceptación globales

- Todos los requerimientos funcionales etiquetados como [Implementado] deben aprobar pruebas de aceptación automatizadas o manuales antes del paso a producción.
- Todos los requerimientos no funcionales con criterio cuantitativo deben ser verificados mediante pruebas reproducibles (carga, regresión, auditoría).
- Ningún cambio en producción debe degradar requerimientos previamente verificados.
- La documentación de la API y de los modelos debe permanecer sincronizada con el código fuente vigente en cada release.

7. Matriz de Trazabilidad

La siguiente tabla relaciona cada requerimiento funcional con el módulo o componente del sistema que lo implementa. Permite verificar la cobertura del diseño respecto a la especificación y orientar pruebas de aceptación dirigidas.

Requerimiento	Módulo backend	Componente frontend
RF-001 Registro	userController.register / express-validator	AuthShellCreative, AuthContext.register()
RF-002 Login	userController.login	AuthShellCreative, AuthContext.login()
RF-003 Logout	userController.logout	AuthContext.logout()
RF-004 Verificación de sesión	Middleware auth (verificación JWT)	AuthContext (efecto de inicialización)
RF-005 Protección por rol	Middleware authorize	Wrappers de ruta /admin/*
RF-006 a RF-010 Gestión de usuarios	userController CRUD	UsersCrudPanel
RF-011 a RF-015 Categorías	categoryController CRUD	CategoriesCrudPanel
RF-016 a RF-019 Subcategorías	subcategoryController CRUD	SubcategoriesCrudPanel
RF-020 a RF-024 Productos	productController CRUD + virtuals del modelo Product	ProductsCrudPanel
RF-025 a RF-030 Catálogo público	productController (filtros, slug, brand, tags)	ProductCatalog, página '/'
RF-031 a RF-033 Perfil	userController.profile	MyProfilePanel
RF-034 a RF-036 Panel admin	Rutas restringidas /api/* con middleware authorize	Layouts y paneles bajo /admin
RF-037 a RF-041 Direcciones	addressController CRUD + setDefault (pendiente)	Componente AddressBook en /profile (pendiente)
RF-042 a RF-046 Carrito	orderController (operaciones sobre estado CART) + orderItemController (pendiente)	Componente CartDrawer / página /cart (pendiente)
RF-047 a RF-053 Pedidos	orderController.checkout / updateStatus / calculateTotal (pendiente)	Página /orders y /orders/:id (pendiente)
RF-054 a RF-059 Pagos	paymentController.processPayment / refund (pendiente)	Página /checkout/payment + integración pasarela (pendiente)

Requerimiento	Módulo backend	Componente frontend
RF-060 a RF-065 Envíos	shipmentController + setTrackingNumber + updateStatus (pendiente)	Componente ShipmentTracker + panel /admin/shipments (pendiente)
RF-066 a RF-069 Facturación	billController + generateReference (pendiente)	Componente InvoiceViewer + panel /admin/bills (pendiente)

7.1 Estado de implementación consolidado

Estado	Significado	Módulos / RF correspondientes
Implementado	El requerimiento está completamente desarrollado y operativo en la versión actual.	RF-001 a RF-036 (módulos 3.1 a 3.8: autenticación, usuarios, catálogo, perfil y panel administrativo)
Parcial	El requerimiento está parcialmente cubierto; existen aspectos pendientes de pulido o validación.	RF-033 (edición avanzada del perfil de usuario)
Pendiente — Especificado	Funcionalidad especificada conforme al diagrama de clases, pendiente de implementación. Forma parte del alcance funcional del producto.	RF-037 a RF-069 (módulos 3.9 a 3.14: direcciones, carrito, pedidos, pagos, envíos y facturación)
Fuera de alcance	Funcionalidad reconocida como necesaria a futuro pero no documentada en este SRS.	Integración con pasarela de pago concreta, facturación electrónica fiscal, integración con operadores logísticos específicos

7.2 Cierre del documento

Este documento constituye la línea base de especificación de requerimientos del sistema KN-Store en su versión actual. Toda modificación posterior deberá registrarse en una nueva revisión, manteniendo la trazabilidad respecto a esta versión 1.0.

Aprobación: el documento se considera vigente a partir de la fecha indicada en la portada y permanecerá así hasta su sustitución por una versión posterior.